Python for Artificial Intelligence

Object-Oriented Programming II: Inheritance & Polymorphism

Week 2, Day 4

Today's Agenda

- Hour 1: Inheritance, overriding, and the different inheritance patterns (multilevel, hierarchical, multiple, hybrid) plus MRO
- Hour 2: Polymorphism, duck typing, and dunder methods (__str__, __eq__, __add__)
- Hour 3: Practice building a small class hierarchy
- By the end: you can share behavior between related classes instead of duplicating it

Hour 1

Inheritance & Overriding Methods

CONCEPT

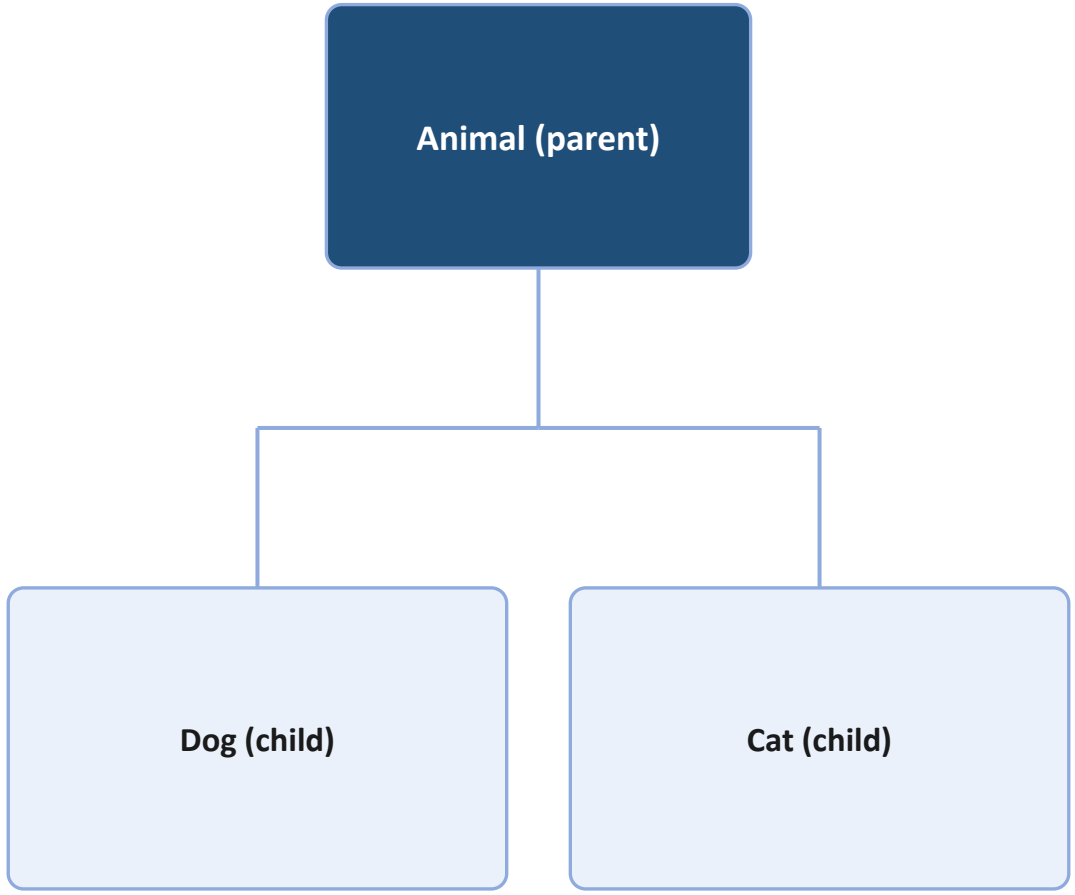
Why Do We Need Inheritance?

- Imagine you have Dog and Cat classes, and both need name, sound(), and eat()
- Writing the same __init__ and eat() twice duplicates code and doubles future bugs
- Inheritance lets a class (the child) reuse and extend another class (the parent)
- The child automatically gets all the parent's attributes and methods, no copy-pasting needed
- You only write what's different in the child class

CONCEPT

Parent and Child Classes

- The parent class (also called the base or super class) defines the shared behavior
- The child class (also called the sub class) inherits from the parent and can add or change behavior
- In Python, a child class is written as class Child(Parent):
- A child 'is a' parent: a Dog is an Animal, a Cat is an Animal



Inheritance Syntax

SYNTAX

```
class Parent:
    def __init__(self, name):
        self.name = name

class Child(Parent):
    pass
```

PARAMETERS

`Parent` the existing class whose attributes and methods will be shared

`Child(Parent)` declares that Child inherits everything Parent has

YOU CAN

✓ Create a Child object and immediately use Parent's methods on it

✓ Add brand new methods that only exist on Child, not on Parent

✓ Override a Parent method by redefining it with the same name inside Child

YOU CANNOT

✗ Use a Child-only method on a plain Parent object

✗ Inherit from a class that hasn't been defined yet earlier in the file

Returns: not applicable; inheritance is a class relationship, not a method call

EXAMPLE

A Basic Parent and Child

Dog inherits __init__ and describe() from Animal without rewriting them.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def describe(self):
        return self.name + " is an animal."

class Dog(Animal):
    pass

d = Dog("Rex")
print(d.describe())  # Rex is an animal.
```

THINK & PRACTICE

Quick Check

Create a parent class Vehicle with __init__ setting self.brand, and a method info(self) returning self.brand + " is a vehicle." Create a child class Car(Vehicle) with no extra code, then test info() on a Car object.

ANSWER

Quick Check: Answer

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

    def info(self):
        return self.brand + " is a vehicle."

class Car(Vehicle):
    pass

c = Car("Toyota")
print(c.info())  # Toyota is a vehicle.
```

CONCEPT

Overriding Methods

- A child class can redefine a parent method with the same name, this is called overriding
- When you call the method on a child object, Python uses the child's version, not the parent's
- This lets each child customize shared behavior while keeping the same method name
- Example: every Animal makes a sound(), but a Dog barks and a Cat meows

EXAMPLE

Overriding sound() in Dog and Cat

Both classes inherit from Animal, but each overrides sound() differently.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return "..."

class Dog(Animal):
    def sound(self):
        return "Woof!"

class Cat(Animal):
    def sound(self):
        return "Meow!"

d = Dog("Rex")
c = Cat("Milo")
print(d.sound())  # Woof!
print(c.sound())  # Meow!
```

THINK & PRACTICE

Think & Practice

Using the Vehicle/Car classes from before, override info(self) in Car so it returns self.brand + " is a car." instead of the parent's generic message. Test it.

ANSWER

Think & Practice: Answer

```
class Car(Vehicle):
    def info(self):
        return self.brand + " is a car."

c = Car("Toyota")
print(c.info())  # Toyota is a car.
```

THINK & PRACTICE

Think & Practice (Harder)

Create a parent class Employee with __init__ setting self.name and self.base_salary, and a method salary(self) that returns self.base_salary. Create a child class Manager(Employee) that overrides salary(self) to return self.base_salary + 20000 (a fixed bonus).

ANSWER

Think & Practice (Harder): Answer

```
class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary

    def salary(self):
        return self.base_salary

class Manager(Employee):
    def salary(self):
        return self.base_salary + 20000

e = Employee("Ali", 50000)
m = Manager("Sara", 50000)
print(e.salary())  # 50000
print(m.salary())  # 70000
```

CONCEPT

Inheritance Comes in Several Shapes

- Everything so far has been single inheritance: one parent, one child
- Real class hierarchies often need more shapes than that
- Four more patterns to know: multilevel, hierarchical, multiple, and hybrid
- The mechanism is always the same (class Child(Parent):); what changes is how many parents/children are involved

CONCEPT

Multilevel Inheritance

- A chain: A is the parent of B, and B is the parent of C
- C inherits everything from B, and everything B inherited from A
- Useful when a category has increasingly specific sub-categories
- Example: BankAccount -> SavingsAccount -> PremiumSavings

EXAMPLE

Multilevel Inheritance

PremiumSavings gets deposit() from BankAccount and add_interest() from SavingsAccount, plus its own cashback().

```
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

class SavingsAccount(BankAccount):
    def add_interest(self, rate):
        self.balance += self.balance * rate

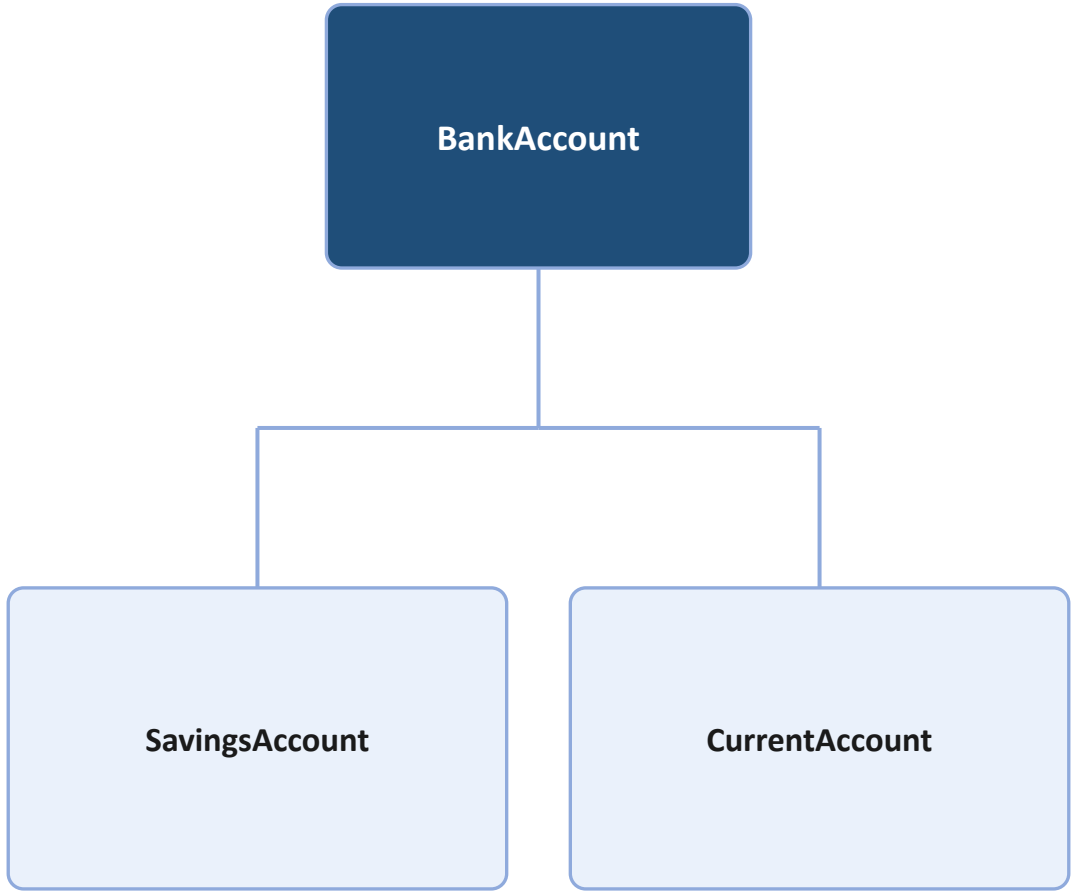
class PremiumSavings(SavingsAccount):
    def cashback(self):
        self.balance += 100

p = PremiumSavings("Ali", 2000)
p.deposit(500)       # from BankAccount
p.add_interest(0.05) # from SavingsAccount
p.cashback()         # from PremiumSavings itself
print(p.balance)
```

CONCEPT

Hierarchical Inheritance

- One parent, multiple children - this is the shape you already used with Animal -> Dog, Cat
- Each child inherits the same parent behavior, then adds its own
- Common when several related categories share a base, but differ from each other
- Example: BankAccount -> SavingsAccount and BankAccount -> CurrentAccount



EXAMPLE

Hierarchical Inheritance

Two independent children of the same parent, each with their own extra method.

```
class SavingsAccount(BankAccount):
    def add_interest(self, rate):
        self.balance += self.balance * rate

class CurrentAccount(BankAccount):
    def overdraft(self, amount):
        self.balance -= amount

s = SavingsAccount("Ari", 3000)
c = CurrentAccount("Bia", 3000)

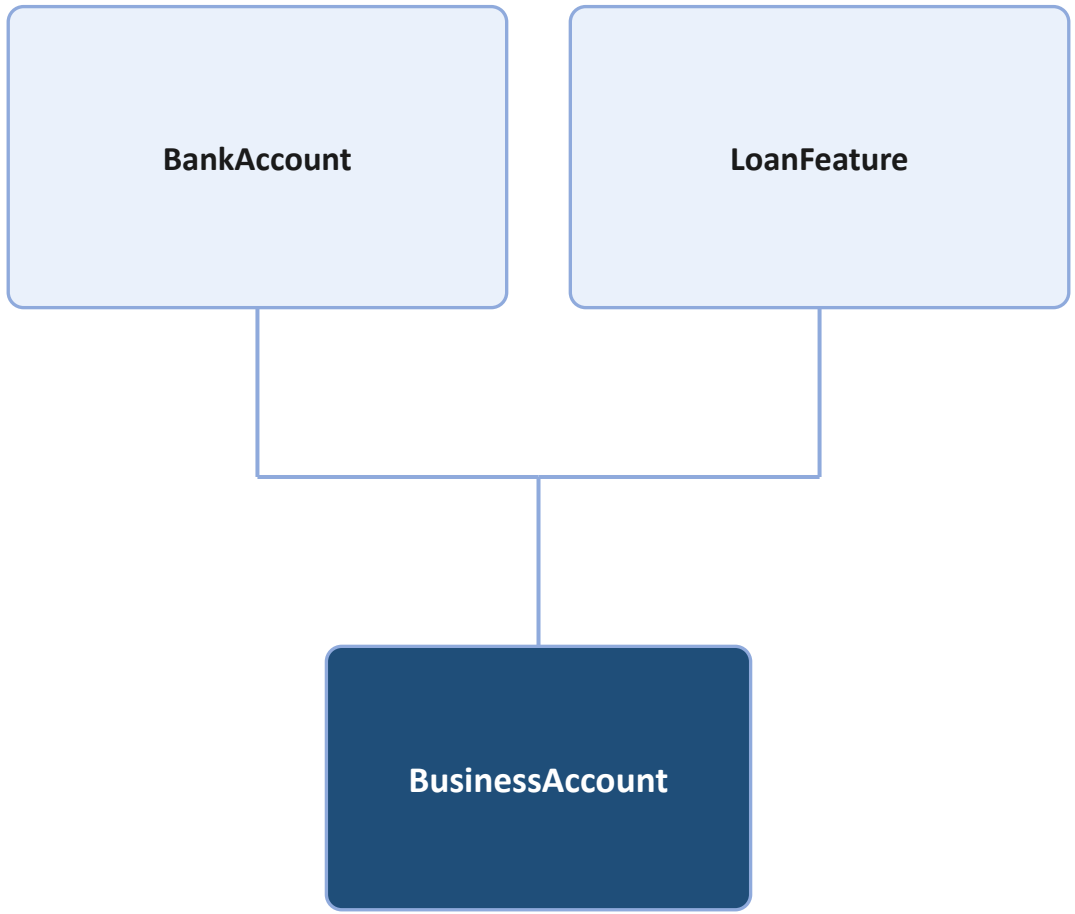
s.add_interest(0.05)
c.overdraft(500)

print(s.balance)
print(c.balance)
```

CONCEPT

Multiple Inheritance

- A single child inherits from two (or more) unrelated parent classes at once
- Written as class Child(Parent1, Parent2):
- Useful for combining two separate capabilities into one class
- Example: BusinessAccount needs both BankAccount behavior and a separate LoanFeature



EXAMPLE

Multiple Inheritance

BusinessAccount combines two unrelated parents into one class.

```
class LoanFeature:
    def apply_loan(self):
        print("Loan Approved")

class BusinessAccount(BankAccount, LoanFeature):
    pass

b = BusinessAccount("CompanyX", 10000)
b.deposit(5000)      # from BankAccount
b.apply_loan()       # from LoanFeature
print(b.balance)
```

CONCEPT

Method Resolution Order (MRO)

- What happens if two parents in multiple inheritance define a method with the same name?
- Python needs a rule to decide which one 'wins' - this rule is the Method Resolution Order
- Simple rule for now: Python searches parents left to right, in the order you listed them
- class C(A, B): - if both A and B define show(), C uses A's version, because A is listed first

EXAMPLE

MRO in Action

A is listed before B, so A's show() wins.

```
class A:
    def show(self):
        print("Show from A")

class B:
    def show(self):
        print("Show from B")

class C(A, B):
    pass

obj = C()
obj.show()   # Show from A (A is listed first)
```

THINK & PRACTICE

Quick Check

In the MRO example above, if you changed the class definition to class C(B, A): instead, what would obj.show() print, and why?

ANSWER

Quick Check: Answer

```
class C(B, A):
    pass

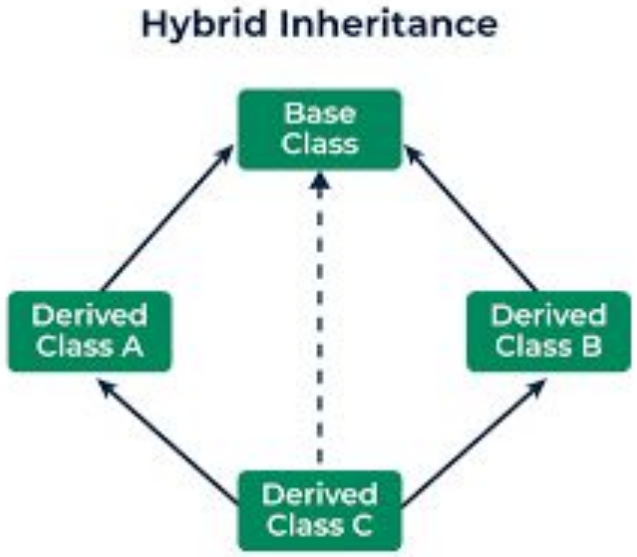
obj = C()
obj.show()   # Show from B

# Python always checks parents in the order they were
# listed. B is now listed first, so B's show() wins.
```

CONCEPT

Hybrid Inheritance

- Hybrid inheritance is simply a combination of more than one inheritance pattern in the same hierarchy
- Example: BankAccount has two children (hierarchical) - SavingsAccount and CurrentAccount
- Then a new class inherits from both of those children at once (multiple)
- It looks complicated, but it's just the patterns you already know, stacked together



EXAMPLE

Hybrid Inheritance

HybridAccount combines hierarchical (two children of BankAccount) with multiple inheritance (inheriting from both).

```
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

class SavingsAccount(BankAccount):
    def add_interest(self, rate):
        self.balance += self.balance * rate

class CurrentAccount(BankAccount):
    def overdraft(self, amount):
        self.balance -= amount

class HybridAccount(SavingsAccount, CurrentAccount):
    pass

h = HybridAccount("Rubaisha", 5000)
h.add_interest(0.05)   # from SavingsAccount
h.overdraft(2000)      # from CurrentAccount
print(h.balance)
```

THINK & PRACTICE

Think & Practice (Harder)

Build a hierarchical + multiple inheritance combo: create Person (name, age), then Student(Person) with a study() method and Employee(Person) with a work() method. Then create WorkingStudent(Student, Employee) and confirm one object can call study(), work(), and access name and age.

ANSWER

Think & Practice (Harder): Answer

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def study(self):
        print(self.name, "is studying")

class Employee(Person):
    def work(self):
        print(self.name, "is working")

class WorkingStudent(Student, Employee):
    pass

w = WorkingStudent("Zara", 21)
w.study()   # Zara is studying
w.work()    # Zara is working
print(w.age)  # 21
```

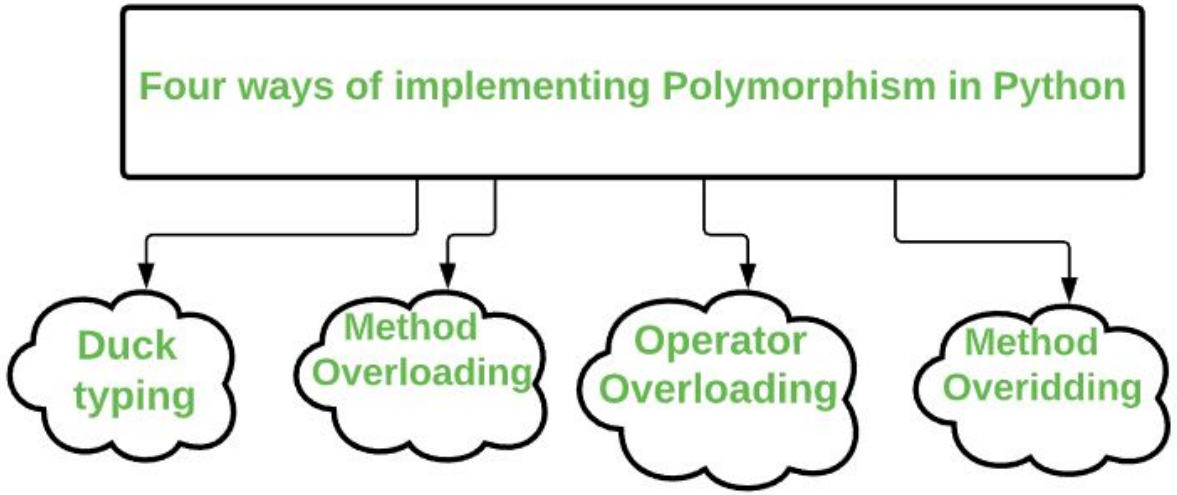
Hour 2

Polymorphism & Dunder Methods

CONCEPT

Polymorphism: One Interface, Many Behaviors

- Polymorphism means: the same method name behaves differently depending on the object it's called on
- We already saw this with sound(): calling it on a Dog and a Cat gave different results
- The real power shows up in a loop: you can call the same method on a list of different objects, and each one responds in its own way
- You don't need to check 'is this a Dog or a Cat' first, you just call .sound() and let each object handle it



EXAMPLE

Polymorphism in a Loop

The loop doesn't care whether each animal is a Dog or a Cat; it just calls .sound().

```
animals = [Dog("Rex"), Cat("Milo"), Dog("Buddy")]

for a in animals:
    print(a.name + ": " + a.sound())

# Rex: Woof!
# Milo: Meow!
# Buddy: Woof!
```

CONCEPT

Duck Typing: Polymorphism Without Inheritance

- "If it walks like a duck and quacks like a duck, treat it like a duck"
- Python doesn't check an object's class before calling a method - it just tries to call it
- So two completely unrelated classes (no shared parent at all) can both work in the same loop, as long as they have the same method name
- This is polymorphism achieved through behavior, not through inheritance

EXAMPLE

Duck Typing

Bird and Airplane share no parent class at all, but both work in lets_fly() because both have a fly() method.

```
class Bird:
    def fly(self):
        print("Bird is flying")

class Airplane:
    def fly(self):
        print("Airplane is flying")

def lets_fly(entity):
    entity.fly()  # works for ANY object with a fly() method

b = Bird()
p = Airplane()

lets_fly(b)   # Bird is flying
lets_fly(p)   # Airplane is flying
```

CONCEPT

Dunder Methods: Customizing Built-In Behavior

- Dunder means 'double underscore', like __init__, which you already know
- Dunder methods let your class hook into Python's built-in behavior: printing, comparing, adding, and more
- __str__(self) controls what print(object) and str(object) display
- __eq__(self, other) controls what happens when you compare two objects with ==
- Without these, Python shows an unhelpful default like <__main__.Dog object at 0x...>

__str__ and __eq__ Syntax

SYNTAX

```
def __str__(self):
    return "some readable text"

def __eq__(self, other):
    return self.attribute == other.attribute
```

PARAMETERS

self the object print() or == was called on

other the second object being compared with ==

YOU CAN

✓ Return a formatted string from __str__ describing the object

✓ Compare specific attributes inside __eq__ to define what 'equal' means for your class

✓ Use print(object) directly once __str__ is defined; Python calls it automatically

YOU CANNOT

✗ Return a non-string value from __str__

✗ Expect == to work sensibly on your objects without defining __eq__ yourself

Returns: __str__ returns a string; __eq__ returns True or False

EXAMPLE

Adding __str__ and __eq__ to Student

Now print(student) and student1 == student2 behave the way we want.

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no

    def __str__(self):
        return "Student(" + self.name + ", roll " + str(self.roll_no) + ")"

    def __eq__(self, other):
        return self.roll_no == other.roll_no

s1 = Student("Ali", 101)
s2 = Student("Ali Khan", 101)

print(s1)          # Student(Ali, roll 101)
print(s1 == s2)    # True (same roll_no)
```

THINK & PRACTICE

Quick Check

Without __str__ defined, what would print(some_object) show? Why does defining __str__ change that?

ANSWER

Quick Check: Answer

```
# Without __str__:
print(s1)
# <__main__.Student object at 0x7f1a2c3b4d90>

# Python's default print() representation shows the
# class name and a memory address, which isn't useful.
# Defining __str__ tells Python exactly what text to
# show instead, because print() calls __str__ for you.
```

CONCEPT

Operator Overloading: __add__

- __str__ and __eq__ hook into print() and ==. Other dunder methods hook into other operators
- __add__(self, other) defines what the + operator does between two objects of your class
- Without it, adding two of your own objects with + raises a TypeError
- This is called operator overloading: the same + symbol behaves differently depending on the object type

__add__ Syntax

SYNTAX

```
def __add__(self, other):
    return some_combined_value
```

PARAMETERS

`self` the object on the left side of the +

`other` the object on the right side of the +

YOU CAN

✓ Return any value that makes sense for your class (a number, a new object, a string)

✓ Combine attributes from both objects however your class needs

YOU CANNOT

✗ Use + on two objects of a class that doesn't define __add__ (raises TypeError)

✗ Expect + to work automatically just because __init__ or __str__ is defined

Returns: whatever you choose to return - there's no fixed rule, it depends on what '+' should mean for your class

EXAMPLE

Overloading + for BankAccount

Adding two accounts together returns their combined balance.

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def __add__(self, other):
        return self.balance + other.balance

acc1 = BankAccount("Ali", 5000)
acc2 = BankAccount("Sara", 7000)

print(acc1 + acc2)  # 12000
```

CONCEPT

A Note on Method Overloading

- In languages like Java/C++, you can define the same method name multiple times with different parameter lists - this is called method overloading
- Python does NOT support this: if you define the same method name twice in a class, the second definition silently replaces the first
- Python fakes overloading-like behavior using tools you already know: default parameters and *args
- Both let one method handle a varying number of arguments, which is what overloading was trying to achieve anyway

EXAMPLE

Faking Overloading with Default Parameters and *args

One add() method handles 1, 2, or 3+ numbers - no overloading needed.

```
class Calculator:
    def add(self, a, b=0, c=0):   # default parameters
        return a + b + c

calc = Calculator()
print(calc.add(5))          # 5
print(calc.add(5, 10))      # 15
print(calc.add(5, 10, 20))  # 35

class Calculator2:
    def add(self, *numbers):     # *args from Day 6
        return sum(numbers)

calc2 = Calculator2()
print(calc2.add(1, 2, 3, 4, 5))  # 15
```

THINK & PRACTICE

Quick Check

Add __add__ to a Point class (with self.x and self.y) so that p1 + p2 returns a new Point whose x and y are the sums of both points' x and y values.

ANSWER

Quick Check: Answer

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return "(" + str(self.x) + ", " + str(self.y) + ")"

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)  # (4, 6)
```

THINK & PRACTICE

Think & Practice (Harder)

Add __str__ and __eq__ to a Point class with __init__ setting self.x and self.y. __str__ should return something like "(3, 4)". __eq__ should return True only if both x and y match on both points.

ANSWER

Think & Practice (Harder): Answer

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return "(" + str(self.x) + ", " + str(self.y) + ")"

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

p1 = Point(3, 4)
p2 = Point(3, 4)
print(p1)          # (3, 4)
print(p1 == p2)    # True
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3

Practice: Building a Class Hierarchy

CONCEPT

Designing a Hierarchy: Look for 'Is a Kind Of'

- If you find yourself thinking 'X is a kind of Y', that's a signal for inheritance
- A Dog is a kind of Animal, a Manager is a kind of Employee, a Circle is a kind of Shape
- Start with the shared attributes and methods in the parent
- Then override only what genuinely differs in each child

WORKED EXAMPLE

Animal, Dog & Cat End to End

A complete small hierarchy combining inheritance, overriding, and polymorphism.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return "..."

    def __str__(self):
        return self.name + " says " + self.sound()

class Dog(Animal):
    def sound(self):
        return "Woof!"

class Cat(Animal):
    def sound(self):
        return "Meow!"

pets = [Dog("Rex"), Cat("Milo")]
for pet in pets:
    print(pet)
```

THINK & PRACTICE

Practice

Build a small hierarchy: a parent class Shape with __init__ setting self.name, and a method area(self) that returns 0. Create child classes Square(Shape) and Circle(Shape), each overriding area() using a side or radius attribute you add in their own __init__.

ANSWER

Practice: Answer

```
class Shape:
    def __init__(self, name):
        self.name = name

    def area(self):
        return 0

class Square(Shape):
    def __init__(self, name, side):
        super().__init__(name)
        self.side = side

    def area(self):
        return self.side * self.side

class Circle(Shape):
    def __init__(self, name, radius):
        super().__init__(name)
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

shapes = [Square("sq1", 4), Circle("c1", 3)]
for s in shapes:
    print(s.name, s.area())
```

NOTE

A Quick Word on super()

- In the answer above, super().__init__(name) calls the parent's __init__ from inside the child
- This avoids rewriting the parent's setup code in every child class
- You'll see super() again in future projects; for now, just recognize what it does

THINK & PRACTICE

Practice (Harder)

Extend the Shape hierarchy with a Triangle(Shape) class using base and height attributes, where area() returns 0.5 * base * height. Also add __str__ to Shape that returns self.name + " has area " + str(self.area()), and confirm it works correctly for all three shapes via polymorphism.

ANSWER

Practice (Harder): Answer

```
class Shape:
    def __init__(self, name):
        self.name = name

    def area(self):
        return 0

    def __str__(self):
        return self.name + " has area " + str(self.area())

class Triangle(Shape):
    def __init__(self, name, base, height):
        super().__init__(name)
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height

t = Triangle("t1", 6, 4)
print(t)  # t1 has area 12.0
```

Home Practice: Interview-Style Questions

- 1. What is inheritance, and what problem does it solve?
- 2. What does it mean to override a method? Give an example.
- 3. Name the four inheritance patterns beyond single inheritance and describe each in one sentence.
- 4. In class C(A, B):, if both A and B define the same method, which one runs and why?
- 5. What is duck typing, and how is it different from polymorphism through inheritance?
- 6. What is polymorphism, and why is it useful in a loop over different object types?
- 7. What does __str__ control, and what happens if you don't define it?
- 8. What does __add__ let you do, and why does Python raise TypeError without it?
- 9. Why doesn't Python support true method overloading, and what two tools fake it?
- 10. Build a class hierarchy of your choice (at least one parent, two children) with one overridden method.

Key Takeaways & Coming Up

- Inheritance lets a child class reuse a parent's attributes and methods without duplicating code
- Beyond single inheritance: multilevel (a chain), hierarchical (one parent, many children), multiple (one child, many parents), and hybrid (a combination) - MRO decides which parent wins on a naming conflict
- Overriding lets a child class customize a specific method while keeping the same name
- Polymorphism means the same method call behaves differently depending on the actual object; duck typing gets this without any inheritance at all
- Dunder methods like __str__, __eq__, and __add__ hook your class into Python's built-in print(), ==, and + behavior
- Coming up in Day 10: Advanced OOP - encapsulation, abstract classes, and dataclasses